

7. Skriv om koden

```
mylist = ['Ho', 'ho', 'ho!']

while mylist:
    x = mylist.pop()
    print(x)

print(mylist)
```

så att listan `mylist` inte är tom när man är klar.

Tips: använd `.copy()`. Hur kan man göra om man inte får använda `copy`?

8. Testa att använda **continue**, **break** och **pass** i **while** loopen nedan. Vad händer?

```
n = 10
while n > 0:
    n -= 1
    if n == 7:
        # test continue, break or pass here
    print(n)
```

9. Skriv en **while** loop som läser in tal från användaren och sparar dem i en lista. Detta ska göras tills användaren skriver in 0 och då ska loopen avslutas och listan skrivas ut.

10. Skriv funktionen

```
def naive_gcd(a,b):
    pass
```

som beräknar GCD av `a` och `b` genom att testa alla tal mellan 1 och `min(a,b)` för att se vilket som är det största talet som delar båda.

11. Skriv funktionen `portkod()` som läser in en siffra (1–9) i taget från användaren. Efter varje siffra är inläst ska funktionen testa om de fyra senaste siffrorna matchar med korrekt kod. Om korrekt kod ges ska funktionen skriva ut `Door unlocked!` och returnera `True`. Välj vad som är korrekt portkod själv, t.ex. 1337.

Tanketips: Vilken loop ska vi använda?

12. Om vi har `lst = [1, 2, 3]` så kan man stoppa in ett element mellan 1:an och 2:an med instruktionen `lst.insert(1, 'x')`. Resultatet är att listan blir `[1, 'x', 2, 3]`.

Ett annat men ekvivalent sätt är att skriva `lst[1:1] = ['x']`.

- Vad är det som händer i den instruktionen?
- Vilket sätt tycker du är tydligast?

Kapitel 4

Uppslagstabeller, filhantering och mer om loopar

En “datastruktur” är en typ för att samla data. Istället för att ha ett tal i en `float` eller en sträng i en `str` så håller en datastruktur reda på många olika data på ett logiskt sätt. Vi har redan sett ett antal sätt i kursen, som till exempel listor. I det här kapitlet ska vi titta på *uppslagstabeller* som har likheter med listor men kan indexeras med andra typer än heltal.

Vi vill också kunna spara data till en fil på datorn eller läsa in data från en fil. Vi ska i kapitlet även titta på hur detta kan göras. Slutligen kommer vi även titta lite mer på vad man kan göra med loopar i Python.

4.1 Uppslagstabeller

Uppslagstabeller (eng. *dictionaries*) är en datastruktur som blivit en arbetshäst för scriptspråk. De är enkla att använda och bra till mycket. Uppslagstabellen är som en lista, men man indexerar med *nycklar* (eng. *keys*) istället för med heltal. Man kan tänka på en uppslagstabell som en mängd av nyckel-värde par (eng. *key-value pairs*). I Python används typen `dict` för uppslagstabeller.

Det finns två typiska sätt i Python att skapa en uppslagstabell. Till exempel kan man skapa en uppslagstabell som har nycklarna 'DA2005' och 'MM2001' med respektive värden 7.5 och 30 genom:

```
hp1 = dict(DA2005=7.5, MM2001=30)
hp2 = {'DA2005': 7.5, 'MM2001': 30}
```

Båda sätten är identiska och `hp1 == hp2`. Lägg märke till att första sättet har samma syntax som att anropa en funktion med nyckelparametrarna DA2005 och MM2001, medan det andra sättet använder sig av en *sträng* för nyckeln och värdet separerat med ett kolon.

För att komma åt ett värde används `[]`-syntax med *nyckelvärdet*:

```
print('Number of ECTS points for DA2005:', hp1['DA2005'])
```

```
Number of ECTS points for DA2005: 7.5
```

Lägg märke till att själva nyckeln också kan vara en variabel. I både `hp1` och `hp2` är nycklarna strängar, dvs av typen `str`. Ett exempel som illustrerar denna poäng:

```

DA2005 = 33
MM2001 = 'hello'
hp3 = {DA2005: 7.5, MM2001: 30}
print('Number of ECTS points for DA2005:', hp3[DA2005])
print('Number of ECTS points for DA2005:', hp3[33])
print('Number of ECTS points for MM2001:', hp3[MM2001])
print('Number of ECTS points for MM2001:', hp3['hello'])

```

```

Number of ECTS points for DA2005: 7.5
Number of ECTS points for DA2005: 7.5
Number of ECTS points for MM2001: 30
Number of ECTS points for MM2001: 30

```

Här har antingen värdet av till exempel MM2001 använts som nyckel direkt, eller så har identifieraren använts. Om man skulle försöka använda sig av en *sträng* av värdet "MM2001" här, så får man ett fel:

```
print('Number of ECTS points for MM2001:', hp3['MM2001'])
```

```
KeyError: 'MM2001'
```

Det här har att göra med att alla objekt i Python som inte kan ändras kan användas som nycklar. Uppslagstabeller implementeras med en teknik som kallas hashtabeller och innebär att man konverterar nyckeln till ett heltal (det "hashas", vilket avser att man "hackar upp" och blandar bytes till ett heltal) som används för uppslagning. Om nyckel kan ändras, "muteras", då finns det risk att man inte hittar tillbaka till det man stoppat in i tabellen. Generellt gäller att *icke-muterbara* värden, till exempel strängar, tal, tupler etc., kan användas som nycklar. Det här utesluter muterbara typer som till exempel listor.

Använder man sig av dict() för att skapa en uppslagstabell, så tolkas nyckelparametrarna alltid som strängar.

För mer detaljerad dokumentation av uppslagstabeller i Python se:

- <https://docs.python.org/3/tutorial/datastructures.html#dictionaries>
- <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>

4.1.1 Viktiga egenskaper

- En uppslagning är billig (dvs snabb).
- Det är enkelt att ändra innehåll.
- Uppslagstabeller är enkla att iterera över (men du får inte ändra i en uppslagstabell under iteration).

4.1.2 Andra namn

- Hashtabell
- Hash map

I "traditionella" språk (C, C++, Java, m.fl.) har de inte samma särställning som i scriptspråken. Scriptspråken har gjort uppslagstabeller lättanvända.

4.1.3 Fler exempel

Tomma uppslagstabellen kan skrivas {}, detta är syntaktiskt socker för dict().

```
empty = {}  
print(empty)  
print(dict())
```

```
{}  
{}
```

Som vi noterat ovan kan man skapa uppslagstabeller på följande sätt:

```
test1 = {'hopp': 4127, 'foo': 4098, 'hej': 4139, 12: 'hej'}  
test2 = dict([('hej', 4139), ('hopp', 4127), ('foo', 4098)])
```

```
print(test1)  
print(test2)
```

```
{'hopp': 4127, 'foo': 4098, 'hej': 4139, 12: 'hej'}  
{'hej': 4139, 'hopp': 4127, 'foo': 4098}
```

Observera att nycklar och värden inte behöver vara av samma typ!

Man kan uppdatera uppslagstabeller på liknande sätt som listor, och lägga till nya nyckel-värde par genom [-] och ta bort med **del**:

```
hp = {}  
hp['DA2005'] = 7  
print(hp['DA2005'])
```

```
hp['DA2005'] += 0.5  
print(hp['DA2005'])
```

```
hp['MM2001'] = 30  
hp['UG1001'] = 1  
print(hp)
```

```
del hp['UG1001']  
print(hp)
```

```
7  
7.5  
{'DA2005': 7.5, 'MM2001': 30, 'UG1001': 1}  
{'DA2005': 7.5, 'MM2001': 30}
```

4.1.4 Iterera över uppslagstabeller

Vi kan använda **for** för att iterera över nycklarna:

```
for key in hp:  
    print(key)
```

```
DA2005  
MM2001
```

Vi kan även använda `.keys()`:

```
for key in hp.keys():  
    print(key)
```

```
DA2005  
MM2001
```

och `.values()`:

```
for val in hp.values():  
    print(val)
```

```
7.5  
30
```

Vi kan även extrahera alla par av nycklar och värden med `.items()`:

```
for key, val in hp.items():  
    print('Number of ECTS points for ' + key + ': ' + str(val))
```

```
Number of ECTS points for DA2005: 7.5  
Number of ECTS points for MM2001: 30
```

Alla dessa tre funktioner returnerar itererbara objekt med element som består av antingen nycklar, värden eller tupler med nyckel-värde-par.

4.1.5 Funktioner för uppslagstabeller

Vi kan även manipulera uppslagstabeller med liknande funktioner som för listor:

```
print(len(hp))  
hp['foo'] = 20.0  
print(len(hp))  
  
print('DA2005' in hp)  
print('20.0' in hp)
```

```
2  
3  
True  
False
```

En till användbar funktion är `.get()`, som tar två argument: nyckelvärdet och ett värde som returneras om nyckeln inte finns i uppslagstabellen. Om nyckeln finns, returneras värdet som hör till nyckeln.

```
print(hp.get('DA2005', 42)) # key exists  
print(hp.get('bar', 42)) # key does not exist
```

```
7.5  
42
```

4.2 Filhantering

Filer innehåller information som ligger på hårddisken, till exempel textfiler, bilder eller filmer, etc. Du kommer åt filer genom att interagera med operativsystemet. Detta kan göras direkt från Python.

4.2.1 Läsa från filer

För att till exempel läsa från en fil `data.txt` skaffar vi först ett “handtag” (eng. *handle*). Det gör man med `open()`, som tar ett filnamn som obligatoriskt argument.

```
h = open('data.txt')
```

Ibland lägger man till ett andra argument med strängen “r” som markerar att man tänker läsa (*read*) filen.

```
h = open('data.txt', 'r') # 'r' för "read"
```

För att läsa in hela filens innehåll till en sträng skriver vi sedan:

```
h.read() # whole file in string
```

eller en lista med strängar där varje list-element är en rad från filen:

```
h.readlines() # whole file as list of strings
```

Detta funkar inte bra i många fall med stora filer. Det blir problem om man läser in en fil som är större än datorns RAM-minne.

För att bara läsa in en rad från `h` använder vi:

```
h.readline()
```

Notera att innehållet i en fil läses *en gång*, så har vi redan använt oss av `read` eller `readlines` så kommer `readline` att inte göra något, eftersom handtaget/filpekaren redan har kommit till slutet av filen.

4.2.2 Skriva till filer

Om vi vill skriva till en fil `out_data.txt` måste återigen använda `open` för att skapa ett filhandtag, men nu måste man skicka med strängen “w” som andra argument (w för *write*) för att indikera att man tänker just skriva till filen.

```
output = open('out_data.txt', 'w')
```

Detta *skapar* eller *tömmer* filen `out_data.txt`. **Varning:** Var försiktig då allt i filen `out_data.txt` kommer att försvinna om den redan finns! Om vi vill skriva till `output` (dvs till filen `out_data.txt`) så använder man idag nyckel-parametern `file` till `print`:

```
print('This is my first line', file=output)
print('\n', file=output) # start a new line
print('This is my second line', file=output)
```

I äldre kod kan du komma att se ett annat skrivsätt:

```
output.write('This is my first line')
output.write('\n') # start a new line
output.write('This is my second line')
```

Här används `w r i t e` som är en funktion som är associerad med en öppnad fil. Det här skrivsättet används också i många andra programmeringsspråk.

Observera att det inte är möjligt att skriva till en fil som är öppnad för läsning, och inte heller tvärtom.

4.2.3 Stänga filer

När vi är klara med en fil bör vi stänga den:

```
h.close()
output.close()
```

Det är viktigt att stänga filer:

- Antalet öppnade filer är begränsat i ett operativsystem – slösa inte med dem!
- Ett annat program kan vilja skriva till öppnad fil – släpp den så annat program inte behöver vänta!
- Du kan förlora data om du inte stängt filen.

När ett program avslutas så stängs öppna filer automatiskt, men det anses ändå vara god programmering att tydligt stänga en fil.

4.2.4 Ett exempel

Tänk dig en fil `people.txt` med komma-separerad information: efternamn, förnamn, födelseår.

```
Svensson,Lisa,1815
Olsson,Erik,1901
Mörtberg,Anders,1986
```

Följande program skapar en fil `modern.txt` med alla från 1900-talet:

```
h_in = open('people.txt', 'r')
h_out = open('modern.txt', 'w')
for n in h_in:    # loop through the file, row by row
    lname, fname, year = n.split(',')
    if int(year) > 1900:
        h_out.write(fname + ' ' + lname + '\n')
h_out.close()
h_in.close()
```

Öppnar vi nu filen `modern.txt` så kommer den innehålla

```
Erik Olsson
Anders Mörtberg
```

4.2.5 Ett praktiskt idiom: `with open`

Python-konstruktionen `with open` har blivit populär och anses idag vara idiomatisk för att arbeta med filer. Anledningen är att idiomat förbättrar läsbarheten för program och uppmuntrar till god kodstruktur.

Här är ett minimalt exempel:

```
with open('ut.txt', 'w') as out:
    print('hej hopp', file=out)
```

Detta motsvarar

```
out = open('ut.txt', 'w')
print('hej hopp', file=out)
out.close()
```

Det finns flera fördelar med att använda `with open`:

- Ditt program blir med lättläst.
- Det uppmuntrar till god kodstruktur.
- Det anses underlätta felhantering.
- Filer stängs när man lämnar `with`-blocket

I Python kallas `with` för en *kontexthanterare* (eng: *context handler*) och kan tillämpas även för annat än filer.

Exemplet ovan kan skrivas om med `with` på följande sätt:

```
with open('people.txt', 'r') as h_in, open('modern.txt', 'w') as h_out:
    for n in h_in:
        lname, fname, year = n.split(',')
        if int(year) > 1900:
            print(f'{fname} {lname}', file=h_out)
```

4.2.6 Att tänka på kring filhantering

- Läs och skriva filer är långsamt.
 - Undvik att läsa en fil två gånger.
 - Använd filer som mellanlagring bara om du måste.
- En del filer är stora. Undvik därför att läsa in allt om det ej behövs. Att läsa in 10 GB data till en dator med 8 GB RAM kan vara katastrofalt för en beräknings hastighet, och om man ändå behandlar data radvis så kan det gå mycket snabbare att läsa in data rad för rad.
- Undvik att hårdkoda sökvägar. Du ska inte anta att andra användare har filerna på samma plats som du.
- Tillfälliga filer bör ha unika och tillfälliga namn.
 - Använd modulen `tempfile` (<https://docs.python.org/3/library/tempfile.html>) för säkerhet och bekvämlighet!

4.2.7 Tre speciella filer

På Unix-system (det inkluderar olika Linux-varianter och MacOS) så finns alltid tre filer öppna, och det har förts över till PC-system också:

- `sys.stdin`
- `sys.stdout`
- `sys.stderr`

Du kommer åt dessa om du har använt instruktionen `import sys` som betyder “läs in modulen `sys`” (läs mer om `import` i kapitel 7!).

Det som är speciellt med dessa filer är att de inte är knutna till någon slags fil på hårddisken (eller motsvarande), åtminstone inte utan annan programmatisk instruktion. I vanliga fall är `sys.stdin` knutet till ditt tangentbord och om du läser från den filen så är det som att läsa av tecken vartefter du skriver

dem. Filerna `sys.stdout` och `sys.stderr` kan man se som knutna till den terminal eller utdata-enhet som din Python-tolk använder. Dessa två kan styras om, till exempel till en datafil, och det är grunden till att man har *två* utdata-filer: om du skriver varningar och felmeddelanden till `sys.stderr` så beblandas inte dessa med utdata. Du får helt enkelt två skilda kanaler för data och användarkommunikation.

Dessa tre filer är ett arv från 70-talets datorer och hör till Unix-traditionen, men är än idag mycket användbart.

4.3 Mer om loopar: nästling

Syntaxen för en **for**-loop bygger på att den indragna “kroppen” på loopen har en eller flera satser. Man kan förstås använda *fler* loopar:

```
for i in range(3):
    for j in range(3):
        print(i * j)
```

```
0
0
0
0
1
2
0
2
4
```

Märk att man ökar indraget för varje loop man gör. Indraget är viktigt för att avgöra strukturen på programmet och hur det exekveras! Ett exempel:

```
print('-----')
for i in range(1,4):
    for j in range(1,4):
        print(i, j, i * j, sep=' ')
    print('-----')
```

```
-----
1  1  1
1  2  2
1  3  3
-----
2  1  2
2  2  4
2  3  6
-----
3  1  3
3  2  6
3  3  9
-----
```

Notera att `print('-----')` endast körs en gång i den yttre loopen. Vad hade hänt om vi istället haft den lika långt indragen som `print(i, j, i * j, sep=' ')`?

Vi kan även kontrollera att endast beräkna jämna produkter med hjälp av villkorssatser i looparna. Notera indenteringsnivån på de två olika **if**-satserna.

```
for i in range(1,8):
    if i % 2 == 1:
        continue
    for j in range(1,8):
        if j % 2 == 1:
            continue
        print(i * j)
```

```
4
8
12
8
16
24
12
24
36
```

4.4 Algoritmiskt tänkande

När man lär sig programmering är en av de stora utmaningarna att skapa sig en djup förståelse för datorn och programmeringsspråket, för hur annars ska man veta vad man ska instruera datorn?

Språket Python är ett *högnivåspråk*, vilket innebär att datorns tekniska detaljer i stor utsträckning abstraherats bort. Det kan hjälpa att känna till vissa tekniska detaljer, se till exempel avsnittet om *datarepresentation*, men i Python kan du komma långt ändå och väsentligen låtsas som att du arbetar i en abstrakt och matematisk miljö utan större tanke på hur din dator faktiskt fungerar.

Man kommer däremot inte ifrån det *algoritmiska tänkandet*: att förstå hur man går från problembeskrivning till en lösning i form av kod. Detta kan vara utmanande eftersom vi till vardags inte är så noggranna med att beskriva hur man gör saker. När du kommunicerar med andra om vad som ska göras i ett projekt eller uppdrag så är det mycket som lämnas outtalat. Människor delar så mycket erfarenhet och bakomliggande kunskap att alla detaljer inte behöver kommuniceras. Du och din dator delar däremot inte mycket erfarenheter; för att få datorn att göra det du vill måste du förklara dig in i minsta detalj.

4.4.1 Recept är inte algoritmer

Recept brukar användas som en analogi för algoritmer och används ibland för att illustrera algoritmiskt tänkande. Analogin är bra som första förklaring av vad en algoritm är, men den kan också leda fel eftersom recept bygger på mycket bakomliggande kunskap. Om du har lärt ut matlagning till ett barn så har du nog märkt att det krävs lite mer än att bara läsa receptet.

Ta till exempel uppgiften att tillaga pannkakor. Här är ett recept (där första delen om ingredienser är borttagen) som de flesta av oss kan följa utan problem.

1. Vispa ut vetemjölet i hälften av mjölken till en slät smet.
2. Vispa i resterande mjölk, ägg och salt.
3. Smält smör i stekpannan och håll ner i smeten.

4. Grädda tunna pannkakor i stekpannan. Circa 1 dl smet per lagg är lagom.

Redan i första punkten är det en massa detaljer som saknas. Ska jag vispa i mjölet innan det hamnar i mjölken, eller är det bättre att dumpa all mjöl i mjölken och sedan börja vispa? Vilket redskap bör användas för vispningen? Och var ska vi arbeta? Man kan ju tänka sig att lägga mjölet på bänken och blanda ner mjölken, men det verkar opraktiskt. Ska man kanske använda mjölkförpackningen? Nja, det är nog bättre att ta fram en ren bunke att arbeta i, men hur stor bör den vara?

Låt oss försöka förbättra första punkten så att receptet blir mer algoritmiskt:

1. Blanda vetemjöl och mjölk så här:
 - a. Ta fram en bunke som har plats för 2 liter och häll i hälften av mjölken.
 - b. Ta fram en visp.
 - c. Häll i mjölet, en matsked i taget, och vispa 20 sekunder mellan varje matsked.
2. Vispa i resterande mjölk, ägg och salt.
3. Smält smör i en stekpanna och häll ner i smeten.
4. Grädda tunna pannkakor i stekpannan. Circa 1 dl smet per lagg är lagom.

Det som åstadkommit är att en bedömning plockats bort (vad är en "slät smet"?) och detaljeringsgraden har ökat, förhoppningsvis så att viss tvetydighet har försvunnit. Skulle en sjuåring kunna följa steg 1?

Punkt 2 och 3 kan göras mer algoritmiska på liknande sätt, men lägg märke till att punkt 4 har lite större utmaningar. Dels presenteras uppgifterna i fel ordning (smeten måste ju hållas i pannen för att kunna gräddas!), dels lämnas bedömningar till läsaren. Istället för att föreslå 1 dl smet så bör vi noggrant förklara "häll i smet tills att hela stekpannan täcks av ett tunt lager, men inte mer, eller smeten tar slut." Det sista villkoret hanterar att smeten sällan räcker till en täcka stekpannans yta för sista pannkakan. Vi måste också förklara att stekpannan ska vara varm (hur varm?), det behövs en stekspade, smeten gräddas tills att undersidan blivit brunaktig och då vänder man och gräddar andra sidan. Och vad innebär "brunaktig" egentligen?

Som du ser blir det mycket att skriva och detaljeringsgraden kan snabbt bli lite frustrerande, men det kan vara vad som krävs för att beskriva för en robot hur man gräddar pannkakor på "traditionellt sätt".

4.4.2 *Top-down, down-up, och att göra rätt*

När vi förbättrade pannkaksreceptet så började vi med en ganska grov beskrivning av vad som ska göras och förfinade första punkten. Det är ett vanligt arbetssätt, särskilt när man är lite osäker på vad som ska åstadkommas, och kallas för att arbeta uppifrån och ner, eller *top-down*. Tanken är att stegvis bryta ner en uppgift i mindre delar, tills att man når en nivå där det går att formulera kod för att lösa deluppgiften.

Den motsatta ansatsen är att arbeta nerifrån och upp, *bottom-up*, genom att identifiera användbar kod som det sedan går att komponera ihop en större helhet med. Om man ska använda pannkaksexemplet så kan man till exempel säga "jag vet hur man blandar mjöl och mjölk, och det kommer ju behövas, så jag började koda den deluppgiften."

Du behöver inte välja mellan *top-down* och *bottom-up*, båda ansatserna är användbara, och det viktiga är att arbeta med uppgiften. Du ska inte ha förväntningen att man funderar på problemet, ser lösningen, och sen skriver den kod som behövs. Så kan det vara ibland, åtminstone för en erfaren programmerare, men det är inget fel att experimentera sig fram och leta lösningar. Kom dock ihåg att skriven kod både kan och bör modifieras efter behov. Nybörjare kan vara nöjda över att skriven kod fungerar och undviker därför ibland att göra ändringar, men det innebär ofta att man skjuter upp problemen och hamnar i svårare problem senare. Det är lätt hänt att göra ett lite olämpligt designval och när det upptäcks så bör man rätta misstaget. Det kan innebära att revidera sin kod om och om igen. En av de viktigaste

insikterna sen programmeringens barndom är just att vi alla gör fel, oavsett hur väl man studerat en uppgift och hur noggrant man planerat, och att det är bra att revidera kod.

Ett annat triviale faktum, som ändå missas ibland, är att det är avgörande att förstå uppgiften. Det är, till exempel, lätt hänt att uppdragsgivare och programmerare menar helt olika saker när en uppgift diskuteras och oavsett hur snyggt programmeraren löser uppgiften (enligt sin egen definition) kommer uppdragsgivaren att vara missnöjd. För att återkomma till pannkaksexemplet så avser antagligen en uppdragsgivare med amerikansk bakgrund inte samma sak med ordet "pancake" som en svensk programmerare som kanske översätter till "pannkaka". Det ligger långt utanför den här kursen att lösa den utmaningen, men det är värt att påpeka att även om du programmerar för egen del så kan du upptäcka att det finns fler aspekter på en uppgift än du förstått från början.

4.4.3 Arbeta med uppgiften!

Några saker att tänka på, som kan uppfattas som triviala, men är värdefulla att gå igenom, särskilt när man jobbar tillsammans med andra:

1. Vad är indata till beräkningen? Till exempel, om du vill räkna antalet ord i en text, hur är texten representerad? Den kanske ges som en enda lång sträng? Eller kanske som en lista av strängar, där varje sträng är en mening? Det är också bra att gå igenom lite extremfall. För exemplet med att räkna ord är det bra att veta om kan texten vara tom, det vill säga ha längd noll, eller finns det alltid minst ett ord? Om vi går tillbaka till pannkaksalgoritmen, vet vi alltid att vi har tillräckligt med ingredienser? Det vore kanske bra att anpassa receptet efter hur många ägg man faktiskt har?
2. Hur ska resultatet se ut? I allmänhet vill vi skriva funktioner som löser uppgifter, se kapitlet om *programstruktur*, och då ska man utgå ifrån att funktionen ska returnera ett värde, men värdet kan bestå av mycket information. Om du skriver en funktion som räknar antalet ord i en text så kan det räcka med ett heltal som svar, men kanske är det mer användbart att returnera en uppslagstabell som associerar ord med ett heltal som beskriver hur många gånger det återkommer i texten?

Det vi pratar om här är alltså start- och slutpunkt för en beräkning. Men hur reder man ut det som kommer däremellan? Det är där själva algoritmen ligger och det är inte alls självklart att den är lätt att komma på. Vissa algoritmiska uppgifter är helt enkelt mycket svåra att förstå och hitta lösningar på. I den här kursen är uppgifterna relativt enkla, i den meningen att gymnasiekunskaper i matematik är tillräcklig förberedelse. Oavsett svårighetsgrad så finns det sätt att arbeta sig fram till en algoritm.

Det går att arbeta sig fram till algoritmer och de behöver inte alls vara komplicerade för att lösa problem som är svåra att beskriva. Här är några enkla tips för att komma framåt.

- Skapa små exempel. Dessa exempel kan vara bra för att experimentera med när man vill komma på hur en algoritm ska se ut, och de kan vara användbara när man vill testa sin kod.
- Hur löser du *egentligen* en uppgift i huvudet? Vissa saker, som addition, är vi så duktiga på att vi slutat tänka på vilka steg man tar. En noggrann genomgång av de steg som du är bekväm med kan ge dig lösningen.
- Hur skulle du lösa uppgiften med penna och papper? Prova dig fram!

Just att arbeta med uppgiften är viktigt för att förstå problemet och det kan vara kontraproduktivt att börja skriva kod med en gång; om du inte förstår problemet så kommer din kod sannolikt vara fel. Å andra sidan kan kodning vara ett sätt att upptäcka att man inte förstått problemet.

Slutligen, se också till att förstå den kod du själv skriver. Man ska inte önska sig fram till fungerande kod. Om du tänker "den här funktionen borde ge mig svaret" eller "jag tror det här fungerar" så ska du bromsa upp: ord som "borde" och "tror" är varningssignaler. Du ska *veta* att koden fungerar eller åtminstone ha god skäl för att tro att den fungerar. Om det känns skakigt så ska du helt enkelt testa. Verifiera att

funktioner gör det du tror de gör. Resonera för dig själv om vad som borde hända i din kod. Förutom att du fastställer vad som fungerar och inte fungerar så är det ett bra sätt att lära sig programmering.

4.5 Representation av data

Ofta behöver man inte fundera på hur data lagras i en dator. När man jobbar med till exempel heltal så räcker det att ha en abstrakt bild av att tal lagras som nollor och ettor, för vi kan ju lätt se hur man kan skriva $x = 7$ och på olika sätt använda att heltalet 7 “finns” i x . Reglerna för hur nollor och ettor bildar heltal kallas *heltalsrepresentation* och är idag standardiserat (men Python avviker, se senare kapitel).

De inbyggda typerna i Python (och andra programmeringsspråk) är lätta att använda utan att bry sig om deras representation, men för mer problemspecifika data (tex personers adresser) och abstrakta idéer (som polynom och andra matematiska begrepp) måste programmeraren tänka efter och hitta sätt att använda språkets datastrukturer och andra finesser för att hitta en lämplig *datarepresentation*. I det här kapitlet diskuterar vi några exempel på hur man kan arbeta med datarepresentation.

4.5.1 Lärandemål

- Representation av heltal, flyttal, strängar, och boolska värden
- Andra data
 - Hur ska man lagra koordinater?
 - Hur kan man lagra polynom?
 - Lagring av datum och tid

4.5.2 Primitiva datatyper

Många data representeras på ett rättframt sätt i programmeringsspråk eftersom det finns inbyggda typer för dem. De mest grundläggande typerna brukar kallas *primitiva datatyper* och i Python är det `bool`, `int`, `float`, `complex`, och `str`. Olika språk har olika primitiva datatyper: i språket C är `char` en datatyp som representerar ett tecken i exakt en byte, och strängar är (ungefär) listor av `char`-värden. Python gör tvärtom, eftersom även ett enskilt tecken ska lagras i en `str`.

Listor och uppslagstabeller kallas *sammansatta datatyper*, eftersom de i någon mening består av primitiva datatyper.

4.5.3 Datastrukturer

En datastruktur är en konstruktion som kan samla flera primitiva datatyper till en enhet. Du har redan stött på listor och uppslagstabeller som är utformade för att samla olika mängder av data så att delarna blir lätt åtkomliga. Dessa datastrukturer är generella och deras syfte är att göra det lätt arbeta med data av variabel storlek, men man kan också se dem som bekväm teknik att indexera eller namnge delar av data. I ett senare kapitel introducerar vi *tupler* (kapitel 6) samt begreppen *klass* och *objekt* som ger oss ett sätt att skapa egna datastrukturer. Klasser används gärna för att göra program mer lättlästa.

4.5.3.1 Representera abstraktioner

De flesta uppfattar det nog inte som så konstigt att begrepp som *namn*, *adress*, och *text* är lämpliga att lagras som strängar. Det kan däremot vara svårare att se hur mer abstrakta begrepp ska representeras.

Som ett första exempel på abstraktioner kan vi ta punkter i planet. Vi kan vilja lagra positioner på en karta, punkter i en figur, eller liknande, och behöver ett sätt att arbeta med positionernas koordinater.

Man kan tänka sig att införa två variabler per punkt, som x och y och sätta $x=0$ och $y=0$ för origo, eller använda två listor x och y om man har många punkter, så origo kanske råkar hamna i första elementen: $x[0]=0$ och $y[0]=0$. Nackdelen med detta sätt att arbeta är att de två koordinater som behöver hanteras ihop ligger åtskilt, i de två variablerna x och y . Det kan ibland vara opraktiskt, men framförallt kan det också öka risken för att man gör fel.

Ett alternativ kan vara att låta koordinater lagras som par, eller snarare som listor med två element. Vi kan då exempelvis definiera $origin = [0, 0]$ eller $points = [[0,0], [1,0], [2,1]]$ och få värden grupperade på ett naturligt sätt. Det gör det lätt att göra enkla operationer. En tilldelning kan till exempel se ut som $origin = points[0]$.

Ett annat alternativ är att använda en uppslagstabell för punkter och initiera $origin = \{'x': 0, 'y': 0\}$. På det här viset kan vi alltså namnge elementen och det blir kanske lite lättare att läsa vad man avser i programmet. Vi skulle till exempel kunna skriva $p['x'] = p['x'] + \text{delta}$ för att justera x -koordinaten något, och det skulle nog vara mer lättförståeligt än motsvarande $p[0] = p[0] + \text{delta}$ om vi använder listor för punkter.

Det vi diskuterar här är hur punkter i planet *representeras*. Vi tar ett mycket abstrakt begrepp (punkter), funderar på vilka data man behöver arbeta med (x och y), och bestämmer hur vi kan använda enkla datastrukturer för den faktiska lagringen.

4.5.4 Exempel: personnummer

I många IT-system behöver man hantera personnummer. Hur representeras de bäst? Eftersom ett personnummer kan anges med 10 eller 12 siffror kan det vara lockande att betrakta det som ett heltal. Till exempel kan personnumret 890201-3286 (fiktivt) representeras som 8902013286. Detta är dock ett stort tal, för stort för att rymmas inom ett 32-bitars heltal. Python klarar detta utan problem, men i de flesta andra programmeringsspråk kan det orsaka svårigheter. En lösning är att dela upp personnumret i två delar, exempelvis `[890201, 3286]`. På så sätt rymmer båda delarna inom 32-bitars gränsen.

Nu kan vi ta ett steg tillbaka och fundera över vad vi faktiskt vill göra med personnummer. Oftast är behovet att kunna jämföra dem – till exempel för att besvara frågan “är det här rätt person?” – och det är sällan nödvändigt att utföra aritmetiska operationer på dem. Därför finns det ingen större fördel med att lagra personnummer som heltal.

Om vi däremot vill verifiera att ett personnummer är korrekt inmatat kan de enskilda siffrorna användas för detta ändamål. Vi kan även vilja försäkra oss om att datumdelen i personnumret är ett korrekt datum. Många programmeringsspråk har bra stöd för datumhantering med särskilda datatyper. (I kapitel 7 introducerar vi *moduler*, och Python har modulen `datetime`, som är användbar för arbete med datum.)

Den som har erfarenhet av personnummer kan dra sig till minnes att Skatteverket använder “samordningsnummer” för personer som inte är folkbokförda i Sverige. Samordningsnummer liknar personnummer men är ändå lätt åtskiljbara, eftersom de bygger på födelsedatumet men har en extra justering: man adderar 60 till dagen i födelsedatumet. Om vi exempelvis omvandlar personnumret 890201-3286 till ett samordningsnummer, skulle det bli 890261-3286. Ett sådant datum kan inte representeras med en vanlig datumtyp, eftersom det inte är ett giltigt kalenderdatum.

Kanske är det enklaste att låta personnummer representeras av strängar?

4.5.5 Exempel: polynom

Om man arbetar med polynom (vilket inte bara matematiker kan behöva göra) så behöver man en representation för dem. Det är inte praktiskt att lagra $4x^2 - 3x + 7$ som en sträng, för hur ska man då

göra beräkningar på det? Ett vanligt sätt att arbeta med polynom är att lagra polynomets koefficienter som tal i en lista, och det mest praktiska är att låta koefficienten för termen med grad i vara på plats i i listan. Det kan kännas som baklänges, men en fördel är att polynomets grad är längden på listan minus ett. Dessutom finns det algoritmiska fördelar.

Några exempel gör det tydligare:

Polynom	Python-representation
$3x^2 - 2x + 1$	[1, -2, 3]
x^4	[0, 0, 0, 0, 1]
$4x^2 + 5x^3$	[0, 0, 4, 5]
$5 + 4x + 3x^2 + 2x^3 + x^4$	[5, 4, 3, 2, 1]

En matematiker kan bli orolig för hur vi representerar noll-polynomet. Är det som [] eller [0]? Det är ett specialfall som man kan behöva hantera, till exempel vid beräkning av ett polynoms gradtal, men det är en detalj som vi inte fördjupar oss i här.

Med representation av polynom som koefficientlistor har vi möjlighet att göra olika operationer. Att addera två polynom kommer att hanteras genom att addera elementen i två koefficientlistor, med viss hänsyn till att polynomen kan ha olika gradtal.

Som exempel kan vi titta på derivering av polynom. Vi vill kunna derivera *symboliskt*, utan att beräkna derivatan i en viss punkt, så resultatet ska bli ett nytt polynom. Hur lärde vi oss i skolan? Jo, vi har lärt oss att konstanttermen försvinner, en term ax^b deriveras till abx^{b-1} , och varje term kan deriveras för sig. Vi kan alltså iterera igenom en koefficientlista, hålla reda på graden på termen som koefficienten representerar, och beräkna nya koefficienter som ab . En enkel algoritmidé för att symboliskt derivera ett polynom p givet som en koefficientlista:

- Skippa koefficienten på index 0.
- Håll reda på aktuellt gradtal.
- Lägg de nya koefficienterna i en ny lista.
- Iterera över koefficienterna från index 1 och beräkna de nya koefficienterna.

Vi kan formulera Python-kod nästan direkt från denna algoritmidé:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial[1:]:
        new_coefficient = term * degree
        result.append(new_coefficient)
        degree += 1
    return result
```

Om man testar funktionen så kan det se ut så här:

```
derivative([1, -2, 3])
```

```
[-2, 6]
```

Det innebär alltså att vi har gått från ett abstrakt polynom till en konkret symbolisk representation, deriverat "maskinellt" till ny koefficientlista, som representerar ett annat abstrakt polynom:

$$3x^2 - 2x + 1 \longrightarrow [1, -2, 3] \longrightarrow [-2, 6] \longrightarrow 6x - 2$$

Vi kan konstatera att med rätt representation kan vi gör avancerade beräkningar på abstrakta begrepp.

4.6 Uppgifter

1. Givet de två listorna nedan

```
a = ['a', 'b', 'c']
b = [1, 2, 3]
```

skapa en uppslagstabell `{ 'a': 1, 'b': 2, 'c': 3 }` genom en loop.

Tips: Det finns flera sätt att åstadkomma detta. Ett sätt är att använda `range` för att få ett index som ni sedan kan använda för att hämta ut element i `a` och `b` med.

2. Skriv kod som tar alla par (x, y) av tal mellan 2 och 20 med $x \neq y$ och skriver ut deras gemensamma delare (GCD).

Tips 1: Använd nästlade loopar och `range`.

Tips 2: Använd funktionen `GCD` från förra kapitlet.

Koden ska till exempel skriva ut följande:

```
x y GCD
2 3 1
2 4 2
2 5 1
2 6 2
2 7 1
2 8 2
# osv...
```

3. Gör hela pannkaksreceptet i avsnitt 4.4.1 mer algoritmiskt!
4. I den här kursen använder vi funktioner för att strukturera koden. Vilka funktioner kan du definiera så att pannkaksreceptet i avsnitt 4.4.1 blir lättare att läsa?
5. Skriv ner en algoritm för att addera två heltal givna som två listor med siffror. Du kan ju addera stora tal med papper och penna, men hur kan algoritmen beskrivas i text? Kan du skriva kod för algoritmen? Denna uppgift är svårare än det låter!
6. I uppgift 2 är det onödigt att skriva ut GCD för till exempel $x, y = 2, 3$ och $x, y = 3, 2$. Gör om koden så att x alltid är mindre än y i raderna som skrivs ut.

Tips: I den inre `for` loopen, vad händer om man itererar från nuvarande värde på x ?

7. I kapitel 3 jobbade vi med funktionen `vowels`. Använd en uppslagstabell som har vokaler i `vs` som nycklar och ett tal som värde (du kan hitta på talen själv). Modifiera funktionen så att den istället byter ut vokaler mot talen.
8. I kapitel 3 jobbade vi med funktionen `split` (given nedan). Ha strukturen i denna kod som hjälp för att skriva en funktion `annotate_word` som tar en sträng och associerar varje ord med ett tal där talet representerar när ordet sist förekom i texten (räknat från 1). Funktionen ska returnera en uppslagstabell med ord som nycklar och tal som värden.


```
def split(s):
    out = []
    term = ''
    for c in s:
        if c == ' ':
            out.append(term)
            term = ''
        else:
            term += c

    # add the final word unless it is empty
    if term != '':
        out.append(term)

    return out
```

Funktionen ska funka så här:

```
print(annotate_word('hej du hej hej du'))
```

```
{'hej': 4, 'du': 5}
```

9. Hur kan vi enkelt få `annotate_word` i uppgift 8 att istället returnera en uppslagstabell med tal som nycklar och ord som värden?
10. Modifiera `annotate_word` i uppgift 8 så att inte bara sista förekomsten sparas i texten, utan alla förekomster. Vilken datastruktur bör vi ha som *värde* i vår uppslagstabell?